Algorithms: Homework 3

Yifan Chen

2026 Fall

Problem 5

(Source: KT book pp. 193, Problem 11.)

Suppose you are given a connected graph $G = (V, E)$, with a cost c_e on each edge e. In an earlier problem, we saw that when all edge costs are distinct, G has a unique minimum spanning tree. However, G may have many minimum spanning trees when the edge costs are not all distinct. Here we formulate the question: can Kruskal's Algorithm be made to find all the minimum spanning trees of G?

Recall that Kruskal's Algorithm sorted the edges in order of increasing cost, then greedily processed edges one by one, adding an edge e as long as it did not form a cycle. When some edges have the same cost, the phrase "in order of increasing cost" has to be specified a little more carefully: we'll say that an ordering of the edges is *valid* if the corresponding sequence of edge costs is nondecreasing. We'll say that a *valid execution* of Kruskal's Algorithm is one that begins with a valid ordering of the edges of G.

For any graph G, and any minimum spanning tree T of G, is there a valid execution of Kruskal's Algorithm on G that produces T as output? Give a proof or a counterexample.

Answer:

We divide the edges into sets with the same cost, then sort the sets by the cost. In any set, the order of edges is arbitrary.

For any MST T, we put the edges in T at the beginning of its set. Therefore, as we follow Kruskal's Algorithm, each set will allow just the edges in T, since the connected components are the same no matter how the edges are ordered within the sets, and the edges of T in a set come first and connect the components without any rejection, while the endpoints of any other edge of the set are already connected.

Problem 6

(Source: KT book pp. 192, Problem 9.)

One of the basic motivations behind the Minimum Spanning Tree Problem is the goal of designing a spanning network for a set of nodes with minimum total cost. Here we explore another type of objective: designing a spanning network for which the most expensive edge is as cheap as possible.

Specifically, let $G = (V, E)$ be a connected graph with n vertices, m edges, and positive edge costs that you may assume are all distinct. Let $T = (V, E')$ be a spanning tree of G; we define the *bottleneck edge* of T to be the edge of T with the greatest cost. A spanning tree T of G is a *minimum-bottleneck spanning tree* if there is no spanning tree T' of G with a cheaper bottleneck edge.

(a) Is every minimum-bottleneck tree of G a minimum spanning tree of G? Prove or give a counterexample.

(b) Is every minimum spanning tree of G a minimum-bottleneck tree of G? Prove or give a counterexample.

Answer:

(a) Consider the following graph:

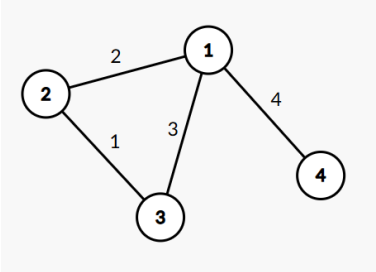


Figure 1: Counterexample.

Then $T = \{(1, 2), (1, 3), (1, 4)\}$ is a minimum-bottleneck tree while it is not an MST.

(b) Any MST T produced by Kruskal's Algorithm is minimum-bottleneck, since the edges with cost less than $\max(T)$ cannot connect the graph. From the previous problem, we know that any MST can be produced by Kruskal's Algorithm, so any MST is a minimum-bottleneck tree.

Problem 7

(Source: KT book pp. 197, Problem 17.)

Consider the following variation on the Interval Scheduling Problem. You have a processor that can operate 24 hours a day, every day. People submit requests to run *daily jobs* on the processor. Each such job comes with a start time and an end time; if the job is accepted to run on the processor, it must run continuously, every day, for the period between its start and end times. (Note that certain jobs can begin before midnight and end after midnight; this makes for a type of situation different from what we saw in the Interval Scheduling Problem.)

Given a list of n such jobs, your goal is to accept as many jobs as possible (regardless of their length), subject to the constraint that the processor can run at most one job at any given point in time. Provide an algorithm to do this with a running time that is polynomial in n. You may assume for simplicity that no two jobs have the same start or end times.

Example. Consider the following four jobs, specified by (start-time, end-time) pairs: (6 P.M., 6 A.M.), (9 P.M., 4 A.M.), (3 A.M., 2 P.M.), (1 P.M., 7 P.M.). The optimal solution would be to pick the two jobs (9 P.M., 4 A.M.) and (1 P.M., 7 P.M.), which can be scheduled without overlapping.

Answer:

Assume there is no job across the midnight; then the problem is the classic interval scheduling problem and can be solved by sorting by the end time.

Otherwise, since there is at most one approved job across the midnight, we can enumerate this job, and the rest problem is the classic problem. Take the maximum of each choice and the variation is solved in polynomial time.

Problem 8

(Source: KT book pp. 203, Problem 29.)

Given a list of n natural numbers $d_1, d_2, \ldots, d_n$, show how to decide in polynomial time whether there exists an undirected graph $G = (V, E)$ whose node degrees are precisely the numbers $d_1, d_2, \ldots, d_n$. (That is, if $V = \{v_1, v_2, \ldots, v_n\}$, then the degree of v_i should be exactly d_i.) G should not contain multiple edges between the same pair of nodes, or "loop" edges with both endpoints equal to the same node.

Answer:

Consider the following polynomial algorithm:

- Choose one vertex x with the max degree d_x.
- If there are not another d_x vertices with remaining degrees, there is no valid construction.
- Otherwise, choose the d_x vertices with the largest degrees, and link them to x.
- Update the degrees, delete the vertices with no remaining degree.
- Repeat until there is no vertex.

Assume at some step the vertex x chooses the vertex u instead of a vertex v of larger degree, and the remaining array still has some solution; then since $1 \leq \deg(u) \leq \deg(v)$, there must be some vertex y which is linked to v and not linked to u, then linking y to u instead of v and linking x to v instead of u will keep the solution.

Therefore, this polynomial algorithm can decide the problem.

Problem 9

In the algorithm for finding strongly connected components, what happens if we modify the algorithm in the following way: we do not compute the transpose of G; instead, we perform another round of DFS, but the outer loop uses the increasing order of the finishing times of all nodes. Is this algorithm correct? Prove its correctness or provide a counterexample.

Answer:

Consider the following graph:

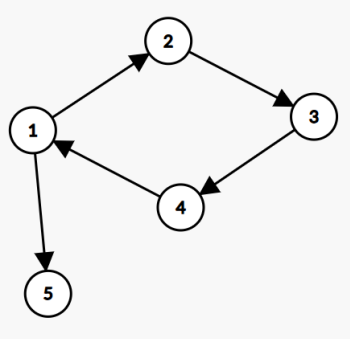


Figure 2: Counterexample.

The first pass starts at 1 and visits 2 before 5, so the earliest finished node is 4. Then 4 will visit all the nodes in the second pass, while 5 is not in its SCC.

AI Disclosure

I used an AI assistant on all problems in this homework, for grammar polishing, LaTeX typesetting, and checking the correctness of my arguments. All mathematical ideas and proofs are my own.